With default parameters, not passing a parameter or passing an undefined one has the same result. But when 'null' is passed, the default value for the parameter is not applied. Remember that 'null' is a valid value. Play around with the above example at *http://www.es6fiddle.net/ii851m5f/*

In addition to assigning values, we can also pass expressions or functions. For example:

```
function simple_interest(p, t=1,
        r=Math.floor(Math.random()*10))
```

The rate of interest is passed as a random number between 0 to 9.

Rest parameters

There are a set of features which were discussed and debated in earlier ECMAScript versions, but never standardised. These have been introduced as a part of ES6. One such feature is rest parameters.

In JavaScript, the *arguments* object is available in every function as a local variable. All parameters passed to a function are stored as *arguments* array in the variable. The use of this object is deprecated and discouraged. For backward compatibility, *arguments* will be still supported in JS engines. In ES6, there is a symbol for rest parameters (...) which helps many values to be passed to a function. A series of values is captured as an array with a name that can be specified in the function definition.

```
1. function add(...nums) {
2.  var result = 0;
3.  nums.forEach(function (num) {
4.    result += num;
5.  });
6.  return result;
7. }
8. add(10, 20, 30, 50);
```

In Line 1, note the new operator, which is three dots. This denotes rest parameters. Note that *nums* is a rest parameter, which is an array. To add numbers, use the array method *forEach* to add each value.

Rest parameters can also be used along with fixed parameters followed by a variable number of parameters. To make the above function more generic, we can make the first argument of the function take the type of operation.

```
function math(operation, ...nums)
```

The first parameter could indicate any operation like addition, multiplication, maximum or minimum.

When using rest parameters, ensure that this parameter is the last argument in the function. If any other parameter follows the rest parameter, a syntax error is thrown. One of the major differences between the rest parameter and *arguments* is that the former is a pure array and it is iterable. This gives a huge advantage when operating on a set of values. This also gives flexibility of fixed parameters and a variable number of parameters.

Spread operator

Another feature related to rest parameters is the spread operator. The rest parameter is used to represent a list of values as one array variable. The spread operator is used to expand an array into individual values in the function call.

```
1. function max(a, b, c) {
2.    // code to find max of three numbers
3. }
4. let a = [10, 20, 30];
5. max(...a); // = max(10, 20, 30)
6. let b = [...a, 40, 50];
```

In Line 5, the function *max* is called with array name using a spread operator. This is equivalent to passing three values individually. Similarly, in Line 6, variable *a* is used with the spread operator to denote the first three elements of array *b*. The spread operator is being used to concatenate arrays.

In previous versions of JavaScript, there was a method *.apply()* for the same purpose. This method is part of *Function. prototype*. The *apply* method takes this as the first argument. The spread operator is simpler and more readable.

In the next issue, we will learn about destructuring. This feature makes extracting subsets of object attributes really simple. It reduces many lines of code to traverse objects and a select few properties. END

The ES6 support matrix

(Green: Full support, Red: No support, Yellow: Partial support)

Feature	Browser			Non-browser	
	Chrome	Firefox	Edge	Node	Babel
Default parameters					
Rest parameters					
Spread operator					

References

[1] Play around with ES6 features online: *http://www.es6fiddle.net/*

[2] Kangax support matrix: *http://kangax.github.io/compat-table/es6/*

By: Janardan Revuru

The author works at the Hewlett Packard Enterprise R&D centre in Bengaluru. He is passionate about JavaScript and related Web frameworks. He can be reached at *janardan.revuru@gmail.com*.